

Programming Assignments

The following are the labs assigned for this semester. The labs are not officially assigned until there is a date associated with them. This is the date that the lab was assigned. Labs are due two weeks after they are defined. Do not submit a lab that you have not tested or **you will lose credit**. If the lab is not satisfactory, I will give you an extra 2 weeks on the deadline to resubmit with no loss of credit. **The additional 2 weeks will be provided if you submit a lab, indicated that it has a problem, and the lab was a good faith effort.** (Keep in mind there is a 3 point deduction from your grade for each lab that is late and a 6 point deduction for each lab that has not been completed.)

If you want to do well in this course, you should work on the labs as soon as they are assigned. They contain important knowledge that will help you do well on the quizzes.

Lab1 9/10/2025

Write a program called rctail which will take a file name as its command line argument and display the last 10 lines in the file. Don't make any assumptions on the size of the file you are displaying. You are to use the UNIX system calls. Make this program as efficient as possible.

Note: In class we will determine how lines of text in a file are delimited.

Lab2 9/17/2025

Write a program that will accept up to 10 file names as command-line arguments. This program will display the names of any file that has been accessed or modified. You should test the files every 2 seconds. Note: if a file has been modified it has also been accessed, so there is no reason to show that a file has been accessed if it has been modified.

Lab3 9/24/2025

Write a program to use putenv to change the TZ value to western time. (California time) Look up how to get the local time and display it before and after the change. Why don't you see the time zone change after you exit the program?

Lab4 10/1/2025

Write a program which will generate a child. The child will have another child, and so on for 10 generations of children. The parent will display its PID and terminate. Each child

will sleep for 1 second before it generates a child, display its PID, display its, parent's PID and terminate. Have the child announce its number that you will assign. **Please be careful with this code.**

Lab 5 10/1/2025

Write a program that will accept the names of 3 processes as command-line arguments. (This can be a single program that you list 3 times in the command line.) Each of these processes will run for as many seconds as: $(PID\%10)*2+3$ and terminate. The parent process will reschedule each child 4 times before giving up. When all children have been scheduled 4 times, the parent will terminate. So your program must keep only one instance of each program running and restart each program up to 4 times when they terminate.

Lab 6 10/8/2025

Write a program will compute prime numbers and write them to a file that will be specified as a command-line argument. The program will display the most recent prime number generated if it receives a SIGUSR1 signal. Also, there will be additional command-line arguments which will specify the times in seconds from the beginning of the run at which the current prime will be displayed. Note: we are allowing multiple times to be in the command-line.

Lab 7 10/8/2025

Find a way to turn on floating point exceptions and write a small program to test this by doing floating point arithmetic and catching the exception. **This is now only an issue when you compile for release.**

Lab 8 10/15/2025 (Assignment date later to give more time)

Write a program that will do the following:

1. It will create two pipes.
2. Create a clone of itself.
3. Parent prompts the user for a string.
4. Parent sends the string over one pipe to the child process.
5. The child process will reverse the characters in the string and send them back to

the parent over the second pipe.

6. The parent will display the string that was sent to it.

Lab 9 10/22/2025

Write a program that will schedule 3 clones. Each clone will communicate to the parent process over a pipe. (i.e. There are 3 pipes.) Each process will write Process $n.t$ (where n is the number of the process 1, 2, or 3 and t is the number of messages that the process n has sent) to the parent 10 times at time intervals of $n*2 + 1$. The parent should display these messages as soon as they are received. Make sure that only one message is received at a time. We will talk about how to do this.

Lab 10 10/29/2025

Write a program to demonstrate how race conditions can occur with shared memory. Your program will do the following:

1. It will allocate a shared memory segment that contains an integer variable and set it to zero.
2. It will create a clone that will start adding 1 to the shared memory segment's integer 100,000,000 times.
3. The parent process will also add 1 to the shared memory segment's integer 100,000,000 times.
4. The parent will make sure that the child process has completed and then it will display the value of integer in shared memory.
5. The parent will then remove the shared memory segment.

Lab 11 11/12/2025

Write two programs. RCSend and RCReceive. Both will have a file name as a command-line argument. RCSend will use messages queues to send the contents of its file to RCReceive. RCReceive will record into the file in its command line. Rcsend and RCReceive will terminate when they have completed their tasks. Think about how to do this. Use message queues.

Lab11 Skip

Write a program to implement a stream oriented queue using shared memory. Make the queue

100K. You should have two functions:

```
int sendQueue( char *buff, int buffsz );    Returns -1 for error and 0 if successful.  
int readQueue(char *buff, int buffsz);      Returns the number of bytes read and -1 if error.
```

Why not a bool return value for the first function? What kind of errors must we handle? How do we set up the queue structure? What are the risks?

Let's show a risk in using shared memory by creating a race condition. How would we do this? We will do it in class.

Lab 12 Skip

Write two programs. RCsend and RCreceive. Both will have a file name as a command-line argument. RCSend will use messages queues to send the contents of its file to RCreceive. RCreceive will record in ito the file in its command line. Rcsend and RCreceive will terminate when they have completed their tasks. Think about how to do this.

Lab13 Skip

Write a program to generate 100,000 prime numbers. You will write this program under the assumption that the program will be run on a computer with 5 CPUs and that your goal is to have this program finish as quickly as possible. You should store the primes in a file which is a command-line argument. We will discuss how to do this program in class.

Lab14 Extra credit 11/19/2025

Using conditional variables and mutexes, write a program to create a queue class that will allow multiple threads to read and write from the queue. We will discuss how to do this in class.

[qtester](#)